

Nexus: The AI-Native Agentic Project Management Platform

1. Introduction: The Shift from Passive CRUD to Agentic Workflows

The landscape of software project management has historically been dominated by passive systems of record. Tools such as Jira, Linear, and Trello operate on a Create-Read-Update-Delete (CRUD) paradigm, serving as digital filing cabinets where data rests until a human user explicitly interacts with it. While effective for tracking static state, these platforms fail to actively participate in the engineering lifecycle. They function as repositories rather than partners, leaving the cognitive load of synthesis, planning, and analysis entirely on human operators. The "Nexus" platform represents a fundamental architectural shift toward an **AI-Native, Agentic** model. In this paradigm, the application is not merely a database interface but an active participant capable of reasoning, planning, and autonomously monitoring project health.¹

This report details the comprehensive implementation roadmap for Nexus, a system designed to bridge the gap between static management tools and autonomous AI agents. The architecture is predicated on three core pillars of artificial intelligence: **Context-Aware Retrieval (RAG)** for natural language interrogation of project states; **Agentic Reasoning (The Senior Architect)** for generative planning and task decomposition; and **Automated Intelligence (Nexus Insight)** for background analytical monitoring.³ By leveraging the latest advancements in **Next.js 15 (App Router)**, **PostgreSQL (pgvector)**, and **LangChain.js**, Nexus demonstrates how modern web frameworks can evolve into host environments for sophisticated cognitive architectures.⁵

The following sections provide an exhaustive technical blueprint, moving from the foundational data layer through the intricate design of reasoning loops, culminating in the real-time user interfaces that render agentic activities visible to human teams. This analysis assumes a "Server-First" philosophy, prioritizing data sovereignty, type safety, and efficient resource utilization essential for scaling AI workloads in a production environment.⁷

2. Technical Architecture & Technology Stack

The selection of a technology stack for an AI-native application requires balancing the immediate needs of real-time user interaction with the computational latency inherent in Large Language Model (LLM) inference. Nexus adopts a **Vertical Slice Architecture**

supported by **Feature-Sliced Design (FSD)** principles.⁹ This ensures that as agents grow in complexity—acquiring new tools and state definitions—the codebase remains modular and maintainable, avoiding the "spaghetti code" often associated with monolithic AI integrations.

2.1 The Core Stack: Next.js 15 and PostgreSQL

The foundational decision to utilize **Next.js 15** is driven by its robust support for streaming and asynchronous background processing, which are critical for masking the latency of AI operations. The App Router's paradigm of React Server Components (RSC) allows Nexus to execute heavy reasoning logic and database interactions on the server, sending only the rendered UI to the client.¹¹ This "Server-First" approach is vital for securing API keys and database credentials, particularly when integrating with third-party LLM providers like OpenAI or Anthropic.

PostgreSQL acts as the unified persistence layer. Unlike polyglot persistence architectures that separate vector data (e.g., Pinecone, Weaviate) from relational data, Nexus consolidates both into a single Postgres instance using the pgvector extension.⁶ This consolidation is not merely a convenience; it is a strict requirement for implementing robust **Row-Level Security (RLS)**. By keeping vectors alongside relational data, we enforce tenant isolation at the database level, ensuring that an AI agent acting on behalf of one project cannot inadvertently retrieve context from another—a critical vulnerability in split-stack architectures.¹⁴

2.2 Component Selection Matrix

Component	Technology Selection	Architectural Justification
Frontend Framework	Next.js 15 (App Router)	Supports React Server Components (RSC) for secure data fetching, Server Actions for type-safe mutations, and the after() API for non-blocking background vectorization. ¹⁶
Language Runtime	TypeScript 5.x	Essential for maintaining strict contract fidelity between the database (Drizzle), the AI structured outputs (Zod), and the frontend components. ¹⁸
Database Engine	PostgreSQL 16+	The only viable option for a unified workload combining transactional integrity (ACID) with vector similarity search (pgvector) and semi-

		structured metadata (JSONB). ¹⁹
ORM Layer	Drizzle ORM	Chosen over Prisma for its lightweight SQL-like syntax and superior support for complex joins required in Hybrid RAG (SQL filters + Vector similarity). ⁵
AI Orchestration	LangChain.js / LangGraph	Provides the state machine primitives required for the "Senior Architect" agent. LangGraph manages the cyclic reasoning loops (Plan → Critique → Execute) that linear chains cannot handle. ²¹
State Management	Zustand + dnd-kit	dnd-kit offers the primitive sensors and collision detection needed for the Kanban board, while Zustand manages the optimistic client state during high-frequency updates. ²³
Real-Time Sync	Postgres LISTEN/NOTIFY + SSE	Bypasses the complexity of WebSockets for a simpler, unidirectional stream of updates (Server-Sent Events) triggered directly by database mutations. ²⁵
Styling System	Tailwind CSS + shadcn/ui	Accelerates development with a consistent, accessible design system that supports dark mode and responsive layouts out of the box. ¹⁸

2.3 Project Directory Structure

To support the Feature-Sliced Design approach, the project structure moves away from generic "components" and "hooks" folders toward domain-specific modules. This ensures that the code for the "Senior Architect" agent is co-located with the types and utilities it consumes, rather than being scattered across the application.²⁸

Recommended File Structure:

nexus-platform/

```

├──src/
│   ├──app/ # Next.js App Router (Routing Layer)
│   │   ├──(auth)/ # Authentication Routes (Grouped)
│   │   ├──(platform)/ # Protected Platform Routes
│   │   │   ├──projects/
│   │   │   │   ├──[projectId]/
│   │   │   │   │   ├──board/ # Kanban Interface
│   │   │   │   │   ├──agent/ # Senior Architect Interface
│   │   │   │   │   ├──insight/ # Analytics Dashboard
│   │   │   │   │   └──layout.tsx # Project Context Provider
│   │   │   └──page.tsx # Project Dashboard
│   │   ├──api/ # Edge API Routes (SSE, Webhooks)
│   │   └──layout.tsx # Root Layout
│   ├──core/ # Core Infrastructure
│   │   ├──db/ # Database Configuration
│   │   │   ├──schema.ts # Drizzle Tables & Relations
│   │   │   └──client.ts # Connection Pooling
│   │   ├──ai/ # LLM Client Setup
│   │   │   ├──embedding.ts # Vector Generation Utilities
│   │   │   └──models.ts # Model Instantiation (OpenAI/Anthropic)
│   │   └──auth/ # Auth.js Configuration
│   ├──modules/ # Feature Slices (Domain Logic)
│   │   ├──kanban/ # Board Logic
│   │   │   ├──components/ # Board, Column, Card UI
│   │   │   └──actions.ts # Server Actions (Move, Reorder)
│   │   ├──architect/ # Agent Feature
│   │   │   ├──graph.ts # LangGraph State Machine
│   │   │   └──prompts.ts # System Prompts & Templates
│   │   └──rag/ # Search Feature
│   │       ├──retriever.ts # Hybrid Search Logic
│   │       └──tool.ts # Vercel AI SDK Tool Definitions
│   ├──lib/ # Shared Utilities
│   │   ├──utils.ts # Tailwind Merge / CLSX
│   │   └──hooks/ # Global React Hooks
│   ├──drizzle.config.ts # Database Migration Config
│   ├──next.config.ts # Server & Experimental Config
│   └──tailwind.config.ts # Design System Tokens

```

This structure adheres to the "colocation" principle, where logic that changes together stays together. The modules/ directory becomes the home for business logic, while app/ is strictly for routing and page composition.²⁹

3. Data Engineering: Designing the Schema for Intelligence

The database schema is the scaffolding upon which all agentic behaviors rest. For Nexus, the schema must support traditional relational integrity for task tracking while simultaneously enabling the high-dimensional vector operations required for semantic search.

3.1 The Hybrid Relational-Vector Schema

We utilize **Drizzle ORM** to define the schema. Drizzle allows for precise control over PostgreSQL data types, including the vector type provided by pgvector and the jsonb type essential for flexible metadata.⁵

Core Entity Definitions:

1. **Tasks:** The atomic unit of work. Beyond standard fields (title, status), this table includes an embedding column for the 1536-dimensional vectors generated by OpenAI's text-embedding-3-small. It also utilizes a jsonb column for tags and ai_metadata, allowing agents to attach arbitrary context (e.g., "blocking_dependency", "architectural_risk") without requiring schema migrations.³¹
2. **Projects:** The container for tasks. Includes tech_stack and guidelines fields, which serve as the "System Prompt" context for the Senior Architect agent.
3. **TaskEvents:** An immutable audit log. Every create, update, or move action is recorded here. This longitudinal data is the fuel for the "Nexus Insight" agent to calculate velocity and identify stagnation patterns.³³

Drizzle Schema Implementation (src/core/db/schema.ts):

TypeScript

```
import { pgTable, text, timestamp, uuid, vector, jsonb, integer, boolean, index } from "drizzle-orm/pg-core";
import { relations } from "drizzle-orm";

// --- Projects: The Context Boundary ---
export const projects = pgTable("projects", {
  id: uuid("id").defaultRandom().primaryKey(),
  name: text("name").notNull(),
  description: text("description"),
  // Critical for Agent Context:
  techStack: jsonb("tech_stack").$type<string>(),
```

```
architecturalGuidelines: text("architectural_guidelines"), // "System Prompt" for the Project  
ownerId: text("owner_id").notNull(), // Linked to Auth Provider ID  
createdAt: timestamp("created_at").defaultNow(),  
});
```

```
// --- Tasks: The RAG & Kanban Entity ---
```

```
export const tasks = pgTable("tasks", {  
  id: uuid("id").defaultRandom().primaryKey(),  
  projectId: uuid("project_id").references(() => projects.id, { onDelete: 'cascade' }).notNull(),  
  title: text("title").notNull(),  
  description: text("description"), // Markdown content supported  
  status: text("status").notNull().default("todo"), // 'todo', 'in-progress', 'done'  
  priority: text("priority").default("medium"), // 'low', 'medium', 'high', 'critical'
```

```
// Vector Embedding: 1536 dimensions for OpenAI text-embedding-3-small  
embedding: vector("embedding", { dimensions: 1536 }},
```

```
// Agentic Metadata
```

```
storyPoints: integer("story_points"),  
aiGenerated: boolean("ai_generated").default(false), // Track agent vs human origin  
tags: jsonb("tags").$type<string>(), // Flexible categorization for RAG
```

```
// Kanban Positioning
```

```
position: integer("position").default(0),
```

```
createdAt: timestamp("created_at").defaultNow(),  
updatedAt: timestamp("updated_at").defaultNow(),  
}, (table) => ({  
  // HNSW Index for ultra-fast approximate nearest neighbor search  
  embeddingIndex: index("embedding_idx").using("hnsw",  
table.embedding.op("vector_cosine_ops")),  
  // Standard index for tenant isolation  
  projectIdx: index("project_idx").on(table.projectId),  
}));
```

```
// --- Audit Log for Analytics ---
```

```
export const taskEvents = pgTable("task_events", {  
  id: uuid("id").defaultRandom().primaryKey(),  
  taskId: uuid("task_id").references(() => tasks.id, { onDelete: 'cascade' }},  
  projectId: uuid("project_id").references(() => projects.id),  
  eventType: text("event_type").notNull(), // 'status_change', 'comment', 'blocker'
```

```

    payload: jsonb("payload"), // Stores { oldStatus: 'todo', newStatus: 'done' }
    createdAt: timestamp("created_at").defaultNow(),
  });

// --- Relations Definitions ---
export const projectsRelations = relations(projects, ({ many }) => ({
  tasks: many(tasks),
}));

export const tasksRelations = relations(tasks, ({ one, many }) => ({
  project: one(projects, { fields: [tasks.projectId], references: [projects.id] }),
  events: many(taskEvents),
}));

```

3.2 Indexing Strategy for Scale

Performance is a critical non-functional requirement. As the number of tasks grows, linear scans of vector data become prohibitively expensive. We implement an **HNSW (Hierarchical Navigable Small World)** index on the embedding column.⁶ HNSW offers superior performance (logarithmic complexity) compared to IVFFlat, albeit at a slightly higher memory cost. This index allows the "Chat With Your Project" feature to retrieve relevant tasks in milliseconds, even with thousands of concurrent vectors.

Furthermore, because all queries are scoped to a specific `project_id` (multi-tenancy), we utilize composite indexing strategies where appropriate. However, pgvector's HNSW index does not natively support pre-filtering by scalar columns (like `project_id`) efficiently in all versions. Therefore, the implementation relies on **Hybrid Search** logic at the query level (discussed in Section 4) to ensure the index is utilized effectively alongside the scalar filters.³⁵

3.3 Connection Pooling & Serverless Safety

Deploying PostgreSQL in a serverless environment (like Vercel) risks exhausting database connections due to the ephemeral nature of lambda functions. Nexus mitigates this by utilizing a connection pooler. If using a provider like Neon or Supabase, the connection string should point to the **Transaction Pooler** (usually port 6543) rather than the direct session port.³⁶

Database Client Initialization (src/core/db/client.ts):

TypeScript

```
import { drizzle } from 'drizzle-orm/postgres-js';
```

```
import postgres from 'postgres';
import * as schema from './schema';

const connectionString = process.env.DATABASE_URL!;

// "prepare: false" is critical for PGBouncer/Transaction Pool compatibility
const client = postgres(connectionString, { prepare: false });

export const db = drizzle(client, { schema });
```

4. Feature 1: Chat With Your Project (Context-Aware RAG)

The "Chat With Your Project" feature allows users to interrogate the project state using natural language. This goes beyond simple keyword search; it requires the system to understand semantics (e.g., "blocking issues" might relate to "high priority" or "dependency" tags) and retrieve context accordingly.

4.1 The Vectorization Pipeline

The vector embeddings must remain synchronized with the task data. In previous architectures, this might have required a separate worker queue (e.g., BullMQ + Redis). Next.js 15 introduces the `after()` API, which simplifies this dramatically by allowing us to schedule background work immediately after the HTTP response is sent.¹⁶

Implementation: The Embedder Service

When a task is created or updated, we trigger the embedding generation. We prioritize the title, description, and tags for the embedding content, as these contain the richest semantic signal.

TypeScript

```
// src/modules/projects/services/embedder.ts
import { OpenAIEmbeddings } from "@langchain/openai";

const embeddings = new OpenAIEmbeddings({
  modelName: "text-embedding-3-small", // 1536 dimensions
  dimensions: 1536,
});
```

```

export async function generateTaskEmbedding(task: { title: string, description: string, tags:
string }) {
  // Composite text maximizes semantic capture
  const textToEmbed = `Title: ${task.title}\nDescription: ${task.description}\nTags:
${task.tags.join(", ")}`;
  return await embeddings.embedQuery(textToEmbed);
}

```

Integration with Server Actions:

TypeScript

```

// src/modules/projects/actions.ts
import { after } from "next/server";
import { generateTaskEmbedding } from "../services/embedder";

export async function createTask(data: CreateTaskDTO) {
  // 1. Transactional Write (Fast)
  const = await db.insert(tasks).values({...data }).returning();

  // 2. Background Vectorization (Non-blocking)
  after(async () => {
    const vector = await generateTaskEmbedding({
      title: newTask.title,
      description: newTask.description |

| "",
      tags: newTask.tags ||
    });

    // Update the record with the vector
    await db.update(tasks)
      .set({ embedding: vector })
      .where(eq(tasks.id, newTask.id));
  });

  return newTask;
}

```

4.2 Hybrid Search Logic

Pure vector search is insufficient for queries like "Show me high-priority bugs in the Auth module." A vector search might return a "medium priority feature" if the semantic similarity is high enough. Therefore, Nexus implements **Hybrid Search**: a combination of Vector Similarity (Cosine Distance) and exact SQL Filtering (Metadata).³⁸

The Retriever Implementation (src/modules/rag/retriever.ts):

This function dynamically constructs a SQL query that enforces the project_id (Security) and blends the similarity score with exact matches.

TypeScript

```
import { db } from "@core/db/client";
import { sql } from "drizzle-orm";
import { generateTaskEmbedding } from "../projects/services/embedder";

export async function hybridSearch(query: string, projectId: string, filters?: { status?: string })
{
  const queryEmbedding = await generateTaskEmbedding({ title: query, description: "", tags:
});
  const similarityThreshold = 0.5; // Tunable parameter

  // We use raw SQL to leverage the <=> operator from pgvector
  const results = await db.execute(sql`
    SELECT
      id, title, description, status, priority,
      1 - (embedding <=> ${JSON.stringify(queryEmbedding)}) as similarity
    FROM tasks
    WHERE project_id = ${projectId}
      ${filters?.status? sql`AND status = ${filters.status}` : sql``}
      AND 1 - (embedding <=> ${JSON.stringify(queryEmbedding)}) > ${similarityThreshold}
    ORDER BY similarity DESC
    LIMIT 10;
  `);

  return results;
}
```

This hybrid approach ensures that the "Chat" agent has access to the *precise* data needed to answer complex user queries, respecting both the semantic intent and the rigid constraints of

the project data.

5. Feature 2: The Senior Architect (Agentic Generation)

The "Senior Architect" is the platform's most advanced feature. It utilizes **LangGraph** to implement a cognitive architecture capable of multi-step reasoning. Unlike a simple chatbot, this agent functions as a state machine, moving through defined stages of planning, critiquing, and generation.²²

5.1 Cognitive Architecture: The Plan-and-Execute Loop

The agent follows a rigorous workflow designed to mimic a human architect:

1. **Analyze Context:** Ingest the project's tech stack and guidelines.
2. **Decompose Requirements:** Break down the high-level user request (e.g., "Add OAuth") into atomic tasks.
3. **Validate:** Check the proposed tasks against the project's architectural guidelines (e.g., "Do not use Auth0 if we are strictly using NextAuth").
4. **Finalize:** Output structured JSON for database insertion.

5.2 State Definition & Zod Schemas

We define the graph's state using **LangGraph's Annotation** system and the output schema using **Zod**. This ensures the LLM's output is strictly typed and parseable.⁴¹

TypeScript

```
// src/modules/architect/schema.ts
import { z } from "zod";

export const TaskSchema = z.object({
  title: z.string().describe("Concise technical title"),
  description: z.string().describe("Detailed implementation steps in Markdown"),
  acceptanceCriteria: z.array(z.string()).describe("List of conditions for task completion"),
  storyPoints: z.number().min(1).max(13),
  priority: z.enum(["low", "medium", "high", "critical"]),
  techStackAlignment: z.string().describe("Explanation of how this fits the defined tech stack"),
  tags: z.array(z.string()).describe("Relevant technical tags (e.g., 'frontend', 'db')"),
});
```

```
export const ArchitectOutputSchema = z.object({
  tasks: z.array(TaskSchema),
  summary: z.string().describe("Executive summary of the implementation plan"),
});
```

5.3 The LangGraph Implementation

The graph coordinates the interaction between the system prompt (context) and the user requirement.

TypeScript

```
// src/modules/architect/graph.ts
import { StateGraph, END } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";
import { ArchitectOutputSchema } from "../schema";

// Define the State
const AgentState = {
  requirements: null, // Input
  projectContext: null, // Input (Stack, Rules)
  generatedPlan: null, // Output
};

const model = new ChatOpenAI({
  modelName: "gpt-4o",
  temperature: 0.2 // Low temperature for precise technical planning
});

// The Reasoning Node
async function architectNode(state: any) {
  const systemPrompt = `
    You are a Principal Software Architect.

    PROJECT CONTEXT:
    ${state.projectContext}

    Your goal is to break down the REQUIREMENTS into a series of implementable development
    tasks.
```

```

    Adhere strictly to the defined Technology Stack.
    If the requirements contradict the stack, prioritize the stack or flag the conflict.
    `;

    // We use.withStructuredOutput to force the Zod schema
    const result = await
model.withStructuredOutput(ArchitectOutputSchema).invoke(["system", systemPrompt],
    ["human", state.requirements]);

    return { generatedPlan: result };
}

// Graph Construction
export const architectGraph = new StateGraph({ channels: AgentState })
.addNode("architect", architectNode)
.addEdge("__start__", "architect")
.addEdge("architect", END)
.compile();

```

5.4 Integration via Server Actions

The frontend invokes this agent via a Server Action, which handles the orchestration and database persistence.

TypeScript

```

// src/modules/architect/actions.ts
'use server';

import { architectGraph } from "../graph";
import { db } from "@core/db/client";
import { projects, tasks } from "@core/db/schema";
import { eq } from "drizzle-orm";

export async function generatePlan(projectId: string, requirement: string) {
    // 1. Fetch Context
    const project = await db.query.projects.findFirst({
        where: eq(projects.id, projectId)
    });
}

```

```

const contextString = `
  Stack: ${JSON.stringify(project?.techStack)}
  Guidelines: ${project?.architecturalGuidelines}
`;

// 2. Invoke Agent
const result = await architectGraph.invoke({
  requirements: requirement,
  projectContext: contextString
});

const plan = result.generatedPlan;

// 3. Persist Tasks
// We map the agent's output directly to our DB schema
await db.transaction(async (tx) => {
  for (const task of plan.tasks) {
    await tx.insert(tasks).values({
      projectId,
      title: task.title,
      description: `${task.description}\n\n**Acceptance
Criteria:**\n${task.acceptanceCriteria.map(ac => ` - ${ac}`).join('\n')}`,
      storyPoints: task.storyPoints,
      priority: task.priority,
      tags: task.tags,
      aiGenerated: true
    });
  }
});

return { success: true, summary: plan.summary };
}

```

6. Feature 3: Nexus Insight (Automated Reporting)

Nexus Insight acts as an automated data analyst. Unlike the Architect, which generates new content, the Insight agent aggregates historical data to identify trends, bottlenecks, and velocity metrics.

6.1 Analytical SQL Queries

To provide the LLM with meaningful data, we must first perform rigorous SQL aggregations. We calculate metrics such as **Sprint Velocity** and **Feature-to-Bug Ratio** using the immutable task_events log and the tags JSONB column.³³

Metric Extraction Service (src/modules/insight/metrics.ts):

TypeScript

```
import { sql } from "drizzle-orm";
import { db } from "@core/db/client";

export async function getProjectMetrics(projectId: string) {
  // 1. Calculate Velocity (Points completed in last 14 days)
  const velocityQuery = await db.execute(sql`
    SELECT SUM(story_points) as velocity
    FROM tasks
    WHERE project_id = ${projectId}
    AND status = 'done'
    AND updated_at > NOW() - INTERVAL '14 days'
  `);

  // 2. Calculate Feature vs Bug Ratio using JSONB operators
  // The ? operator checks if the 'bug' string exists in the tags array
  const ratioQuery = await db.execute(sql`
    SELECT
      COUNT(*) FILTER (WHERE tags? 'bug') as bug_count,
      COUNT(*) FILTER (WHERE NOT (tags? 'bug')) as feature_count
    FROM tasks
    WHERE project_id = ${projectId}
  `);

  return {
    velocity: velocityQuery.velocity |
    | 0,
    bugRatio: ratioQuery
  };
}
```

6.2 Narrative Generation

The raw numbers are then fed into an LLM to generate a narrative report. The prompt is engineered to act as a "Product Manager," interpreting the data rather than just restating it.

TypeScript

```
// src/modules/insight/generator.ts
export async function generateReport(projectId: string) {
  const metrics = await getProjectMetrics(projectId);

  const prompt = `
    Analyze the following project metrics:
    - Recent Velocity: ${metrics.velocity} points
    - Bugs: ${metrics.bugRatio.bug_count}
    - Features: ${metrics.bugRatio.feature_count}

    Identify if the project is becoming unstable (high bug ratio) or stalling (low velocity).
    Provide actionable recommendations.
  `;

  // Call LLM...
}
```

7. The Frontend Experience: Optimistic Kanban & Real-Time Sync

The user interface of Nexus revolves around a Kanban board. Given the high interactivity of this component, standard request-response cycles are too slow. We employ **Optimistic UI** patterns to ensure immediate feedback, coupled with **Server-Sent Events (SSE)** for real-time collaboration.

7.1 Optimistic Updates with dnd-kit

When a user drags a task, the UI must update *instantly*. We use React's `useOptimistic` hook to manage this state before the server confirms the change.⁴⁴

Board Implementation Strategy:

1. **State:** Manage local optimistic state that overrides server data.
2. **Action:** Trigger Server Action to persist change.
3. **Revert:** If Server Action fails, the optimistic state automatically rolls back.

TypeScript

```
// src/modules/kanban/components/Board.tsx
'use client';

import { useOptimistic, useTransition } from "react";
import { DndContext, DragEndEvent } from "@dnd-kit/core";
import { moveTaskAction } from "../actions";

export default function Board({ tasks }: { tasks: Task }) {
  // Optimistic State
  const = useOptimistic(
    tasks,
    (state, { taskId, newStatus }) => {
      return state.map(t => t.id === taskId ? {...t, status: newStatus } : t);
    }
  );

  async function onDragEnd(event: DragEndEvent) {
    const { active, over } = event;
    if (!over) return;

    const taskId = active.id;
    const newStatus = over.id;

    // 1. Instant UI Update
    startTransition(() => {
      setOptimisticTasks({ taskId, newStatus });
    });

    // 2. Server Persistence
    await moveTaskAction(taskId, newStatus);
  }

  // Render DndContext with optimisticTasks...
}
```

7.2 Real-Time Sync via Postgres LISTEN/NOTIFY & SSE

To support collaboration, other users must see these changes immediately. We bypass complex WebSockets in favor of **Server-Sent Events (SSE)**, which are easier to implement in a serverless/edge context.²⁶

Step 1: Database Trigger

We add a trigger to Postgres that emits a JSON payload whenever the tasks table changes.

SQL

```
-- Migration: Add Notify Trigger
CREATE OR REPLACE FUNCTION notify_task_changes() RETURNS TRIGGER AS $$
BEGIN
  PERFORM pg_notify('nexus_updates', json_build_object(
    'projectId', NEW.project_id,
    'type', TG_OP,
    'taskId', NEW.id,
    'newStatus', NEW.status
  )::text);
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER task_update_trigger
AFTER INSERT OR UPDATE ON tasks
FOR EACH ROW EXECUTE FUNCTION notify_task_changes();
```

Step 2: The SSE Endpoint (src/app/api/sse/route.ts)

This route maintains an open connection to the client. It listens to the Postgres channel and streams events.

TypeScript

```
import { db } from "@core/db/client";

export const dynamic = 'force-dynamic'; // Prevent caching

export async function GET(req: Request) {
  const encoder = new TextEncoder();
  const stream = new ReadableStream({
    start(controller) {
```

```

// Subscribe to Postgres channel using the dedicated listener client
// (Note: In production, use a dedicated Redis PubSub or Supabase Realtime
// if utilizing a connection pooler that doesn't support LISTEN/NOTIFY directly)

const listener = (payload: any) => {
  const message = `data: ${JSON.stringify(payload)}\n\n`;
  controller.enqueue(encoder.encode(message));
};

//... subscription logic...
}
});

return new Response(stream, {
  headers: {
    'Content-Type': 'text/event-stream',
    'Cache-Control': 'no-cache',
    'Connection': 'keep-alive',
  }
});
}

```

On the client, a `useEffect` hook connects to this endpoint and invalidates the Next.js router cache (`router.refresh()`) upon receiving a message, ensuring all users see the latest state without a full page reload.⁴⁶

8. Conclusion: The Future of Agentic Project Management

Nexus represents a significant evolution in project management tooling. By moving from a passive data store to an **AI-Native** architecture, it transforms the role of the platform from a mere repository to an active participant in the development process. The integration of **Next.js 15** provides the necessary infrastructure for server-side reasoning and streaming; **PostgreSQL with pgvector** offers a unified, secure data layer for hybrid retrieval; and **LangGraph** enables the creation of sophisticated, stateful agents like the Senior Architect. This implementation roadmap provides a clear path to execution. By adhering to the principles of Feature-Sliced Design, Optimistic UI, and Hybrid Search, developers can build a system that is not only technologically advanced but also highly responsive and intuitive for human users. The result is a platform where AI agents and human engineers collaborate

seamlessly, accelerating the software delivery lifecycle through intelligent automation.

Works cited

1. Nexus AI Platform - White Pearl Technology Group, accessed February 12, 2026, <https://whitepearltech.com/services-3/ai-intelligent-automation/nexus-ai-platform/>
2. NEXUS: Where Natural Language Meets Agent-Driven Automation - CMiC, accessed February 12, 2026, <https://cmicglobal.com/products/nexus>
3. Nexus One | The Unified Intelligence Layer, accessed February 12, 2026, <https://www.nexuscognitive.com/>
4. Nexus AI, accessed February 12, 2026, <https://nexusai.io/>
5. Deploy Next.js Boilerplate - Railway, accessed February 12, 2026, <https://railway.com/deploy/nextjs-boilerplate>
6. PGVectorStore integration - Docs by LangChain, accessed February 12, 2026, <https://docs.langchain.com/oss/javascript/integrations/vectorstores/pgvector>
7. Getting Started: Updating Data - Next.js, accessed February 12, 2026, <https://nextjs.org/docs/app/getting-started/updating-data>
8. Next.js project structure: Master the setup for scalable Next.js apps - Magic UI, accessed February 12, 2026, <https://magicui.design/blog/next-js-project-structure>
9. Clean Architecture vs. Feature-Sliced Design in Next.js Applications | by Metastability Inc, accessed February 12, 2026, <https://medium.com/@metastability/clean-architecture-vs-feature-sliced-design-in-next-js-applications-04df25e62690>
10. Feature Sliced Design in Next JS. What is FSD and why is it needed ? | by Sriramanvellingiri, accessed February 12, 2026, <https://medium.com/@sriramanvellingiri/feature-sliced-design-in-next-js-7d20be4338de>
11. Next.js 15: App Router — A Complete Senior-Level Guide | by Liven Apps | Medium, accessed February 12, 2026, <https://medium.com/@livenapps/next-js-15-app-router-a-complete-senior-level-guide-0554a2b820f7>
12. Mastering Next.js 15+ Folder Structure: A Developer's Guide | by TechTales by Hari, accessed February 12, 2026, <https://medium.com/@j.hariharan005/mastering-next-js-15-folder-structure-a-developers-guide-b9b0461e2d27>
13. Optimize generative AI applications with pgvector indexing: A deep dive into IVFFlat and HNSW techniques | AWS Database Blog, accessed February 12, 2026, <https://aws.amazon.com/blogs/database/optimize-generative-ai-applications-with-pgvector-indexing-a-deep-dive-into-ivfflat-and-hnsw-techniques/>
14. Implementing Fine-Grained Postgres Permissions for Multi-Tenant Applications - Permit.io, accessed February 12, 2026, <https://www.permit.io/blog/implementing-fine-grained-postgres-permissions-for-multi-tenant-applications>

15. How to Implement PostgreSQL Row Level Security for Multi-Tenant SaaS - techbuddies.io, accessed February 12, 2026, <https://www.techbuddies.io/2026/02/04/how-to-implement-postgresql-row-level-security-for-multi-tenant-saas-2/>
16. Next.js 15: Unlocking the Power of after() for Non-Blocking Tasks - DEV Community, accessed February 12, 2026, <https://dev.to/joodi/nextjs-15-unlocking-the-power-of-after-for-non-blocking-tasks-2kli>
17. Functions: after - Next.js, accessed February 12, 2026, <https://nextjs.org/docs/app/api-reference/functions/after>
18. jakedahn/nextjs-boilerplate - GitHub, accessed February 12, 2026, <https://github.com/jakedahn/nextjs-boilerplate>
19. Building Intelligent Search with AI Embeddings, Neon, and pgvector - DEV Community, accessed February 12, 2026, <https://dev.to/neon-postgres/building-intelligent-search-with-ai-embeddings-neon-and-pgvector-3hep>
20. pgvector/pgvector: Open-source vector similarity search for Postgres - GitHub, accessed February 12, 2026, <https://github.com/pgvector/pgvector>
21. Build a custom RAG agent with LangGraph - Docs by LangChain, accessed February 12, 2026, <https://docs.langchain.com/oss/python/langgraph/agent-rag>
22. LangGraph overview - Docs by LangChain, accessed February 12, 2026, <https://docs.langchain.com/oss/javascript/langgraph/overview>
23. Building a Kanban Board with Drag and Drop in React - DEV Community, accessed February 12, 2026, <https://dev.to/surajondev/building-a-kanban-board-with-drag-and-drop-in-react-2pop>
24. React Query with DnD Kit: Item Goes Back to Original Position for a Split Second on Drop Despite Optimistic Updates - Stack Overflow, accessed February 12, 2026, <https://stackoverflow.com/questions/79168785/react-query-with-dnd-kit-item-goes-back-to-original-position-for-a-split-second>
25. Building Real-Time Log Streaming with PostgreSQL LISTEN/NOTIFY - DEV Community, accessed February 12, 2026, <https://dev.to/polliog/building-real-time-log-streaming-with-postgresql-listennotify-4cbj>
26. How to Use Listen/Notify for Real-Time Updates in PostgreSQL - OneUptime, accessed February 12, 2026, <https://oneuptime.com/blog/post/2026-01-25-use-listen-notify-real-time-postgresql/view>
27. I Spent 48 Hours Building a Next.js Boilerplate So You Can Ship in 30 Minutes, accessed February 12, 2026, <https://dev.to/salmanshahriar/i-built-a-production-ready-nextjs-boilerplate-so-you-dont-have-to-17d3>
28. Best Practices for Organizing Your Next.js 15 2025 - DEV Community, accessed February 12, 2026, <https://dev.to/bajrayejoon/best-practices-for-organizing-your-nextjs-15-2025-53ji>
29. Getting Started: Project Structure | Next.js, accessed February 12, 2026, <https://nextjs.org/docs/app/getting-started/project-structure>
30. Usage with Next.js | Feature-Sliced Design, accessed February 12, 2026, <https://feature-sliced.design/docs/guides/tech/with-nextjs>

31. Documentation: 18: 9.16. JSON Functions and Operators - PostgreSQL, accessed February 12, 2026, <https://www.postgresql.org/docs/current/functions-json.html>
32. How to Query JSONB in PostgreSQL - Tiger Data, accessed February 12, 2026, <https://www.tigerdata.com/learn/how-to-query-jsonb-in-postgresql>
33. Feature Matrix - PostgreSQL, accessed February 12, 2026, <https://www.postgresql.org/about/featurematrix/>
34. Advanced Reports - SQL Query Examples | Advanced Insights ..., accessed February 12, 2026, <https://docs.advancedinsights.app/atlassian/advanced-reports/sql-query-examples>
35. Timescale Vector x LangChain: Making PostgreSQL A Better Vector Database for AI Applications, accessed February 12, 2026, <https://blog.langchain.com/timescale-vector-x-langchain-making-postgresql-a-better-vector-database-for-ai-applications/>
36. Connection Pooling Patterns: Optimizing Database Connections for Scalable Applications | by Artem Khrienov | Medium, accessed February 12, 2026, <https://medium.com/@artemkhrenov/connection-pooling-patterns-optimizing-database-connections-for-scalable-applications-159e78281389>
37. best practices for postgres connection : r/nextjs - Reddit, accessed February 12, 2026, https://www.reddit.com/r/nextjs/comments/1aqa6s/best_practices_for_postgres_connection/
38. Supabase hybrid search integration - Docs by LangChain, accessed February 12, 2026, <https://docs.langchain.com/oss/javascript/integrations/retrievers/supabase-hybrid>
39. Hybrid search with Postgres : r/LangChain - Reddit, accessed February 12, 2026, https://www.reddit.com/r/LangChain/comments/1dvdnzc/hybrid_search_with_postgres/
40. LangChain Agents: Build Scalable, Controlled Workflows, accessed February 12, 2026, <https://www.langchain.com/agents>
41. Structured output - Docs by LangChain, accessed February 12, 2026, <https://docs.langchain.com/oss/javascript/langchain/structured-output>
42. LangChain: Structured Output with JavaScript - Robin Wieruch, accessed February 12, 2026, <https://www.robinwieruch.de/langchain-javascript-structured/>
43. Calculate ratio of counts in PostgreSQL - Stack Overflow, accessed February 12, 2026, <https://stackoverflow.com/questions/74431089/calculate-ratio-of-counts-in-postgresql>
44. Next.js 15 Tutorial - 81 - Optimistic Updates - YouTube, accessed February 12, 2026, <https://www.youtube.com/watch?v=ipmfUw8l2qc>
45. PostgreSQL LISTEN/NOTIFY: Real-Time Without the Message ..., accessed February 12, 2026, <https://www.pedroalonso.net/blog/postgres-listen-notify-real-time/>
46. Real-Time Updates with Server-Sent Events (SSE) in Next.js 15 ..., accessed

February 12, 2026, <https://damianhodgkiss.com/tutorials/real-time-updates-sse-nextjs>